

Matrices

1. Diagonal Matrix
2. Lower Triangular Matrix
3. Upper Triangular Matrix
4. Symmetric Matrix
5. Tridiagonal Matrix
6. Band Matrix
7. Topetitz Matrix
8. Sparse Matrix

Diagonal Matrix

Take care of it

	1	2	3	4	5
1	3	0	0	0	0
2	0	7	0	0	0
3	0	0	4	0	0
4	0	0	0	9	0
5	0	0	0	0	6

** IF all element except diagonal are zero, then it is waste of memory & also time (if perf. found) so it can be stored in 1D array

5x5
 $M[i, j] = 0 \text{ IF } i \neq j$
 5 = 10 bytes

5x5 = 25
 25x2 = 50 bytes
 memory

3 7 4 9 6
 0 1 2 3 4 element from A[i-1]

void set (int A[], int i, int j, int x)

For setting element in 1D matrix

```

if (i == j)
    A[i-1] = x;

int get (int A[], int i, int j)
{
    if (i == j) return A[i-1];
    else return 0;
}
    
```

For getting element from matrix

→ #include <stdio.h>

struct Matrix

{
int A[10]; → you can also create in heap
int n;
using malloc

};

void set (struct Matrix *m, int i, int j, int x)

{

if (i == j)

→ A[i-1] = x;

}

int Get (struct Matrix m, int i, int j)

{

if (i == j)

return m.A[i-1];

else

return 0;

}

void Display (struct Matrix m)

{

int i, j;

for (i = 0; i < m.n; i++)

{

for (j = 0; j < m.n; j++)

{

if (i == j)

printf("%d", m.A[i]);

else

printf("0 ");

printf("\n");

}


```

int main()
{
    struct Matrix m;
    m.n = 4;

```

```

    set(m, 1, 1, 5); set(m, 2, 2, 8); set(m, 3, 3, 9); set(m, 4, 4, 12);

```

```

    Display(m); → 5 0 0 0
                  0 8 0 0
8 ← printf("%d\t", Get(m, 3, 2)); 0 0 9 0
                  0 0 0 12
    return 0;
}

```

C++ Class For Diagonal Matrix

	1	2	3	4	5
1	3	0	0	0	0
2	0	7	0	0	0
3	0	0	4	0	0
4	0	0	0	9	0
5	0	0	0	0	6

→

	1	2	3	4	5
1	3	7	4	9	6
2	0	1	2	8	4
3	0	0	1	3	5
4	0	0	0	1	2
5	0	0	0	0	1

$M[i, j]$

IF ($i = j$)

$A[i-1]$

5x5

$M[i, j] = 0$ IF $i \neq j$

Class Diagonal

```

{
    Private:
        int n;
        int *A;

```

Public:

Diagonal (int n)

```

{
    This → n = n;

```

```

    A = new int[n];

```

```

}

```

Page No. _____
Date: / /

```
void set (int i, int j, int x) {
```

```
    int get (int i, int j) {
```

```
        void Display () {
```

```
            ~ Diagonal () & delete [] A; {
```

```
        }
```

```
void Diagonal :: set (int i, int j, int x) {
```

```
    if (i == j) {
```

```
        A[i-1] = x;
```

```
    }
```

```
int Diagonal :: get (int i, int j) {
```

```
    if (i == 1) return A[i-1];
```

```
    else return 0;
```

```
}
```

```
void Diagonal :: Display () {
```

```
    for (i = 0; i < n; i++) {
```

```
        if (i == 1) {
```

```
            cout << A[i-1];
```

```
        } else {
```

```
            cout << "0";
```

```
        }
```

```
    }
```

```
    cout << endl;
```

```
}
```

```
}
```


int main()

{

Diagonal d(4):

d.set(1,1,5);

d.set(2,2,8);

d.set(3,3,9);

d.set(4,4,12);

return;

}

d.Display(); →

5	0	0	0
0	8	0	0
0	0	9	0
0	0	0	12

Lower Triangular Matrix

	1	2	3	4	5
1	a ₁₁	0	0	0	0
2	a ₂₁	a ₂₂	0	0	0
3	a ₃₁	a ₃₂	a ₃₃	0	0
4	a ₄₁	a ₄₂	a ₄₃	a ₄₄	0
5	a ₅₁	a ₅₂	a ₅₃	a ₅₄	a ₅₅

	Row-major														
A	a ₁₁	a ₁₂	a ₁₃	a ₁₄	a ₁₅	a ₂₁	a ₂₂	a ₂₃	a ₂₄	a ₂₅	a ₃₁	a ₃₂	a ₃₃	a ₃₄	a ₃₅
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
rows	row 1			row 2			row 3			row 4			row 5		

$$M[i, j] = 0 \text{ if } i < j$$

$$M[i, j] = \text{non-zero} \text{ if } i \geq j$$

elements

$$\text{Non-zero} = 1 + 2 + 3 + 4 + 5$$

$$= 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

elements

$$\text{Zero} = n^2 - \frac{n(n+1)}{2} = \frac{n(n-1)}{2}$$

$$\text{index}(A[4][5]) = [1+2+5] + 2 = 8$$

$$\text{index}(A[5][4]) = [1+2+3+4] + 3 = 13$$

$$\text{index}(A[i][j]) = \left[\frac{i(i-1)}{2} \right] + j - 1$$

Formula for storing elements in 1D matrix

Lower Triangular Matrix Column-Major Mapping

i \ j	1	2	3	4	5
1	a_{11}	0	0	0	0
2	a_{21}	a_{22}	0	0	0
3	a_{31}	a_{32}	a_{33}	0	0
4	a_{41}	a_{42}	a_{43}	a_{44}	0
5	a_{51}	a_{52}	a_{53}	a_{54}	a_{55}

a_{11}	a_{21}	a_{31}	a_{41}	a_{51}	a_{22}	a_{32}	a_{42}	a_{52}	a_{33}	a_{43}	a_{53}	a_{44}	a_{54}	a_{55}
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Col 1					Col 2				Col 3			Col 4		Col 5

$$\text{Index}(A[4][4]) = [5+4+3] + 0 = 12$$

$$\text{Index}(A[5][4]) = [5+4+3] + 1 = 13$$

$$\text{Index}(A[5][5]) = [5+4] + 2 = 11$$

$$\text{Index}(A[i][j]) = [n + n-1 + \dots + n - (j-2)] + (i-j)$$

$$= [n(j-1) - (1+2+\dots+j-2)] + (i-j)$$

$$= \left[n(j-1) - \frac{(j-2)(j-1)}{2} \right] + (i-j)$$

Formula for storing using column major mapping

→ #include <stdio.h>
#include <stdlib.h>

struct Matrix

{
int *A;
int n;

};

void set(struct Matrix *m, int i, int j, int x)

{
if (i >= j)

$$m \rightarrow A[i * (i-1) / 2 + j - 1] = x;$$

}

For column major

$$m \rightarrow A[n * (j-1) + (j-2) * (j-1) / 2 + i - j] = x;$$

int Get (struct Matrix m, int i, int j)

{
if (i == j)

return m.A[i * (m.n - 1) / 2 + j - 1];

else
return 0;

}

return m.A[m.n * (j - 1) + (j - 2) * (m.n) / 2 + i - j];

void Display (struct Matrix m)

{

int i, j;

for (i = 1; i <= m.n; i++)

{

for (j = 1; j <= m.n; j++)

{

if (i == j)

printf("%d", m.A[i]);

else

printf("0");

}

printf("\n");

}

}

int main()

{

struct Matrix m;

int i, j;

printf("Enter Dimension");

scanf("%d", &m.n);

Page No.
 Date: / /

```
m.A = (int *) malloc (m*n * (m*n-1)/9 * sizeof(int));
```

```
printf("Enter all elements");
```

```
for (i=1; i<=m*n; i++)
```

```
{
```

```
for (j=1; j<=m*n; j++)
```

```
{
```

```
scanf("%d", &x);
```

```
set(m, i, j, x);
```

```
}
```

```
}
```

```
printf("\n\n");
```

```
Display(m);
```

```
}
```

Enter Dimension

Enter all elements

2 0 0 0

1 3 0 0

1 2 3 0

1 2 3 4

2 0 0 0

1 3 0 0

1 2 3 0

1 2 3 4

Let the user
Feel that he
is entering
all the
elements

C++

✓

Revise



Class LowerTri

{

Private:

int *A;

int n;

Public:

LowerTri()

{
n = 2;

A = new int[2*(2+1)/2];

}

LowerTri(int n)

{

This->n = n;

A = new int[n*(n+1)/2];

}

~LowerTri()

{
delete[] A;

}

void set(int i, int j, int x)

int Get(int i, int j)

void Display()

{

void LowerTri::set(int i, int j, int x)

{
if(i >= j)

A[i*(i-1)/2 + j - 1] = x;

}

```
int LowerTri :: Get (int i, int j)
{
    IF (i >= j)
        return A[i * (i-1) / 2 + j - 1];
    return 0;
}

void LowerTri :: Display()
{
    For (int i = 1; i <= n; i++)
    {
        For (int j = 1; j <= n; j++)
        {
            IF (i >= j)
                cout << A[i * (i-1) / 2 + j - 1] << " ";
            else
                cout << "0 ";
        }
        cout << endl;
    }
}

int main()
{
    int d;
    cout << "Enter Dimensions";
    cin >> d;

    LowerTri m(d);

    int x;

    cout << "Enter All elements";
}
```



```

For (int i=1; i<=d; i++)
{
    For (int j=1; j<=d; j++)
    {
        cin >> x;
        Im.set(i,j,x);
    }
    Im.Display();
}
return 0;

```

1	0	0	0	0
1	2	0	0	0
1	2	3	0	0
1	2	3	4	0
1	2	3	4	5

Upper Triangular Matrix

	$j \rightarrow$	1	2	3	4	5
$i \downarrow$	1	a_{11}	a_{12}	a_{13}	a_{14}	a_{15}
2		a_{22}	a_{23}	a_{24}	a_{25}	
3		0	0	a_{33}	a_{34}	a_{35}
4		0	0	0	a_{44}	a_{45}
5		0	0	0	0	a_{55}

5x5
n x n

Row major

A	a_{11}	a_{12}	a_{13}	a_{14}	a_{15}	a_{22}	a_{23}	a_{24}	a_{25}	a_{33}	a_{34}	a_{35}	a_{44}	a_{45}	a_{55}
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Index (A[4][5]) = [5+4*5] + 1 = 25

Index (A[i][j]) = [n + n-1 + n-2 + ... + n-(i-2)] + (j-i)

$$= \left[\frac{(i-1)n - (i-2)(i-1)}{2} \right] + (j-i)$$

↓
Formula

Column-major

	1	2	3	4	5
1	a_{11}	a_{12}	a_{13}	a_{14}	a_{15}
2	0	a_{22}	a_{23}	a_{24}	a_{25}
3	0	0	a_{33}	a_{34}	a_{35}
4	0	0	0	a_{44}	a_{45}
5	0	0	0	0	a_{55}

5x5

5x5

a_{11}	a_{12}	a_{13}	a_{14}	a_{15}	a_{22}	a_{23}	a_{24}	a_{25}	a_{33}	a_{34}	a_{35}	a_{44}	a_{45}	a_{55}
col ₁	col ₂	col ₃	col ₄	col ₅										

$$\text{Index}(A[i][j]) = [1+2+3+4] + 8 = 18$$

$$\text{Index}(A[i][j]) = [1+2+3 + \dots + j-1] + i$$

$$= \left[\frac{j(j-1)}{2} \right] + i - 1$$

Symmetric Matrix

	1	2	3	4	5
1	2	2	2	2	2
2	2	3	3	3	3
3	2	3	4	4	4
4	2	3	4	5	5
5	2	3	4	5	6

5x5

$$M[i][j] = M[j][i]$$

it can be stored in either using

- ① Lower Triangular Matrix
- OR ② Upper Triangular Matrix

Tridiagonal Matrix

	1	2	3	4	5
1	a_{11}	a_{12}	0	0	0
2	a_{21}	a_{22}	a_{23}	0	0
3	0	a_{32}	a_{33}	a_{34}	0
4	0	0	a_{43}	a_{44}	a_{45}
5	0	0	0	a_{54}	a_{55}

Main diagonal $i-j=0$

Lower " $i-j=1$

Upper " $i-j=-1$

$$|i-j| \leq 1$$

$$M[i, j] = \text{non-zero} \quad \text{IF } |i-j| \leq 1$$

$$M[i, j] = 0 \quad \text{IF } |i-j| > 1$$

No. of non-zero
element =
 $5 + 4 + 4$
 $n + n - 1 + n - 1$

Mapping $\rightarrow A$

a_{01}	a_{02}	a_{03}	a_{04}	a_{11}	a_{12}	a_{13}	a_{14}	a_{15}	a_{21}	a_{22}	a_{23}	a_{24}	a_{25}
0	1	2	3	4	5	6	7	8	9	10	11	12	
Lower Diag				Main diagonal					Upper Diag				

$3n-2$

Index ($A[i][j]$)

Case 1:- IF $i-j=1$ index = $j-1$

Case 2:- IF $i-j=0$ index = $n-1+i-1$

Case 3:- IF $i-j=-1$ index = $2n-1+i-1$

Square Band Matrix.

	1	2	3	4	5	6	7	8
1					0	0	0	0
2						0	0	0
3							0	0
4								0
5	0							
6	0	0						
7	0	0	0					
8	0	0	0	0				

Toeplitz Matrix $\swarrow$ elements in a diagonals are same

	1	2	3	4	5
1	2	3	4	5	6
2	7	2	3	4	5
3	8	7	2	3	4
4	9	8	7	2	3
5	10	9	8	7	2

5x5

* It is sufficient to store only one row & only one column element

$$M[i, j] = M[i-1, j-1]$$

element $\rightarrow 0+0-1 = -1$

A	2	3	4	5	6	7	8	9	10
	0	1	2	3	4	5	6	7	8

Index (A[i][j])

Case 1 IF $i \leq j$ index = $j-1$

Case 2 IF $i > j$ index = $n+i-j-1$

Menu Driven Program For Matrices

Menu

1. Create
2. Get $\rightarrow$ Getting at particular index
3. Set $\rightarrow$ setting at particular index
4. Display

choices: 1

void main()

{ int *A, n, ch, x;

printf("Enter dimension");

scanf("%d", &n);

A = (int*) malloc (n * sizeof(int));

do

{ // Display Menu

switch(ch)

{ case 1: For (i=1; i<=n; i++)

{ scanf("%d", &A[i-1]); break;

case 2: printf("Enter indices");

scanf("%d %d", &i, &j);

if (i == j) printf("%d", A[i-1]); else printf("0");

case 3: printf("Enter row, col & ele");

scanf("%d %d %d", &i, &j, &x);

if (i == j) A[i-1] = x; break;

case 4: For (i=1; i<=n; i++)

{ For (j=1; j<=n; j++)

{ if (i == j) printf("%d", A[i-1]); else printf("0");

printf("\n");

while

1. Diagonal [\]

A[i-1]

2. Lower Triangular [\]

Row-major

A[i * (i-1)/2 + j-1]

column-major

A[(j-1)*n - (j-2)*(j-1)/2 + i-1]

3. Upper Triangular [/]

Row-

A[(i-1)*n - (i-2)*(i-1)/2 + j-1]

col

A[j * (j-1)/2 + i-1]

4. Symmetric [\]

Menu Driven Program For Matrices Using Function

```
void create (int A[], int n)
{
    int get (int x[], int i, int j) &
    void set (int x[], int i, int j, int x) &
    void display (int x[], int n) &
}
```

```
int main()
{
    int *A, n, i, j;
    A = new int[n];
    do
    {
        create(A, n);
        while(1);
    }
}
```

How to write c++ class For all Matrices

```
class Diagonal
{
    private:
        int *A, int n;
    Diagonal(int n);
    void create();
    int Get (int i, int j);
    void set (int i, int j, int x);
    void Display();
    ~Diagonal();
}
```

Gr

Diagonal :: Diagonal (int n)

{ This $\rightarrow n = n;$

A = new int [n];

{ $\rightarrow$ change for other Matrices

Diagonal :: ~Diagonal ()

{ delete [A];

$\rightarrow$ * Destructor releases all the resources & heap is also resource so here it is releasing heap.

Void Diagonal :: create ()

{ int x;

For (i=1; i<=n; i++)

{

For (j=1; j<=n; j++)

{

cin >> x;

If (i==j)

A[i-1][j-1] = x;

$\rightarrow$ * change for other Matrices

Void Diagonal :: Display ()

{ For (i=1; i<=n; i++)

{

For (j=1; j<=n; j++)

{

If (i==j)

cout << A[i-1][j-1] << " ";

else

cout << "0";

cout << endl;

$\rightarrow$ change for other Matrices

Vector of Vector

→ `vector<vector<int>> vec;`

`for (int i=0; i<ROW; i++)`

`vector<int> v1;`

`for (int j=0; j<COL; j++)`

`v1.push_back(num)`

`num++`

`;`

`vec.push_back(v1);`

`;`

Q. Given a matrix $m \times n$. If an element is 0, set its entire row and column to 0's and return the matrix.

You must do it in place.

Example 1:

1	1	1
1	0	1
1	1	1

 $\Rightarrow$

1	0	1
0	0	0
1	0	1

 $\rightarrow$

0	1	2	0
3	4	5	2
1	3	1	5
///	///	///	///

 $\rightarrow$

0	0	0	0
0	4	5	0
0	3	1	0
///	///	///	///

→ • A straightforward solution using $O(mn)$ space is probably a bad idea

⇓

• A simple improvement uses $O(m+n)$ space, but still not the best solution

⇓

$O(1)$ space

Ex:-

0	1	1	1	1
1	1	0	1	1
2	1	1	0	1
3	0	0	0	1

Take auxiliary space in matrix

0	0	0	1
0	0	0	0
0	0	0	0
0	0	0	0

```
int m = matrix.size();
int n = matrix[0].size();
```

```
for (int i=0; i<m; i++)
```

```
{
    for (int j=0; j<n; j++)
```

```
{
    if (matrix[i][j] == 0)
```

```
{
    if (j==0)
```

```
{
    col-0 = 0;
```

```
matrix[i][0] = 0;
```

```
else
```

```
matrix[i][0] = 0;
```

```
matrix[0][j] = 0;
```

```
for (int i=m-1; i>=0; i--)
```

```
{
    for (int j=n-1; j>=0; j--)
```

```
{
    if (j==0)
```

```
{
    if (matrix[i][0] == 0 || col-0 == 0)
```

```
{
    matrix[i][j] = 0;
```

```
else
```

```
{
    if (matrix[i][0] == 0 || matrix[0][j] == 0)
```

```
{
    matrix[i][j] = 0;
```

For creating vector of size of 5 row. matrix

```
vector<vector<int>> v(5)
```

Q. Rotate Matrix [inplace]

1	2	3	7	4	1
4	5	6	8	5	2
7	8	9	9	6	3

* intuition $\rightarrow$ 741 is reverse of 147

1	4	7
2	5	8
3	6	9

reverse

5	1	9	11	15	13	2	5
2	4	8	10	14	3	4	1
13	3	6	7	12	6	8	9
15	14	12	16	16	7	10	11

1. Transpose

2. Reverse every row

```
void rotate (vector<vector<int>> & m)
```

```
for (int i=0; i<n; i++)
```

```
for (int j=0; j<i; j++)
```

```
swap (m[i][j], m[j][i]);
```

```
for (int i=0; i<n; i++)
```

```
reverse (m[i].begin(), m[i].end());
```

Q. Search in a 2D Matrix

10	20	30	40
11	21	36	43
25	29	39	50
50	60	70	80

target = 25

Brute Force Approach $\rightarrow O(n^2)$

$\downarrow$

$O(n \log n)$

$\downarrow$

Better approach

target = 25

10	20	30	40
11	21	36	43
22	29	39	50
50	60	70	80

target = 42

10	20	30	40
11	21	36	43
25	29	39	50
50	60	70	80

int i = 0, j = m - 1;

while (i < j & j >= 0)

if (mat[i][j] == x)

cout << "Found at " << i << ", " << j << "

return 1;

{

if (mat[i][j] > x)

j--;

else

i++;

Q7

Pow(x, n)

Implement pow(x, n) which calculates x raised to the power n (i.e. x^n).

Ex 1: $x = 2.0000$, $n = 10$ $x = 2.1000$, $n = 3$, $x = 2.0000$, $n = -2$

O/P: 1024.000

O/P: 9.26100, O/P: 0.250

for (1 - n)

ans = ans * x

Break for

if n = (-)

$x^{-n} = 1/x^n$

$$2^{10} = (2 \times 2)^5 = (4)^5 = 1024$$

$$4^5 = 4 \times 4 \times 4 \times 4 \times 4 = 256$$

$$4^4 = (4 \times 4)^2 = (16)^2 = 256$$

$$16^2 = (16 \times 16)^1 = 256^1$$

$$256^1 = 256 \times (256)^0$$

Optimised

$[n \% 2 == 0] \rightarrow (x * x)$
 $(n/2)$

TC $\rightarrow \log_2 n$

$n \% 2 == 1 \rightarrow ans = ans * x$
 $n = n - 1$

double myPow(double x, int n)

double ans = 1.0;

long long nn = n;

if (nn < 0) nn = -1 * nn;

while (nn)

if (nn % 2)

ans = ans * x;

nn = nn - 1;

{

else

x = x * x;

nn = nn / 2;

}

{

if (n < 0) ans = (double)(1.0) / double(ans);

return ans;

}

Unique pat

Q1

A robot
The
Ho

start			

I/P: ans =

O/P

	0	1	2
0	5		
			5

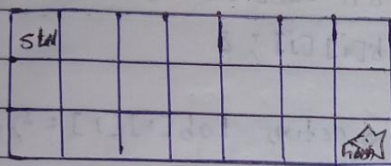
C^0
 $x(2)$

Unique path(I)

Q1

→ 100%

A robot is located at the top-left corner of a $m \times n$ grid (marked 'start' in diagram below). The robot can move only down or right. How many possible unique paths are there?



I/P: $m=3, n=7$

O/P 28

I/P: $m=3, n=2$

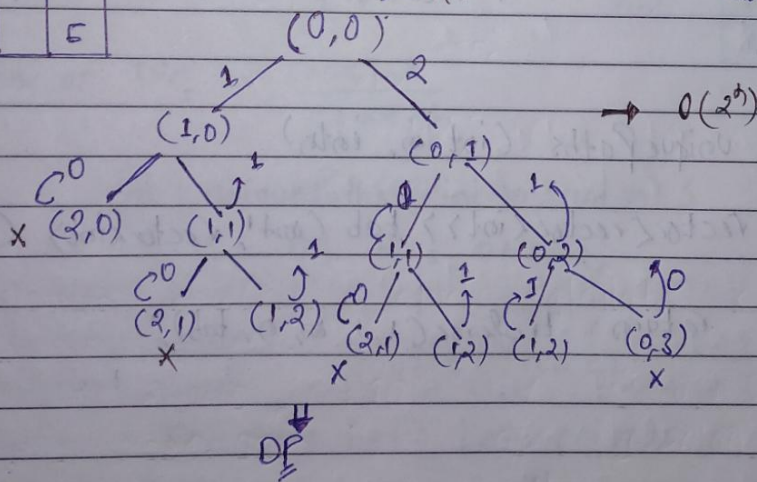
3

I/P: $m=8, n=8$

6



Brute Force :- (use recursion) Try out all possible paths
= 3 Ans +



ans) /

```
int traverse (int i, int j, int m, int n,
              vector<vector<int>> & tab)
```

```
{
```

```
    if ((i <= m && j <= n) && tab[i][j] != -1)
```

```
        return tab[i][j];
```

```
    if (i == m && j == n) return tab[i][j] = 1;
```

```
    if (i > m || j > n) return 0;
```

```
    return tab[i][j] = (traverse(i+1, j, m, n, tab)
                       + traverse(i, j+1, m, n, tab));
```

```
}
```

```
int uniquePaths (int m, int n)
```

```
{
```

```
    vector<vector<int>> tab (m+1, vector<int> (n+1, -1));
```

```
    return traverse(1, 1, m, n, tab);
```

```
}
```

Out-of-the-box
soln
Obsv - 2

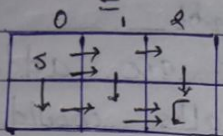
0	1
1	2

Ans

Code of

Out-Of-the-box
solⁿ

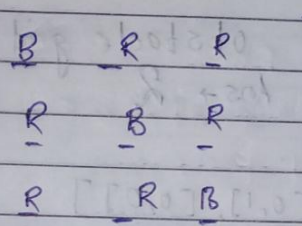
Obsⁿ -2



2 → rightward

1 → downward

$$(m-1 + n-1) = m+n-2$$



3C₂

3C₁

Ans → $m+n-2$

C_{m-1}

or

$m+n-2$

C_{n-1}

Code of ${}^{10}C_3 \rightarrow \frac{10 \times 9 \times 8}{3 \times 2 \times 1}$

int uniquePaths (int m, int n) {

int N = m+n-2;

int r = m-1;

double res = 1;

for (int i=1; i<=r; i++)

{

res = res * (N-r+i)/i;

return (int) res;

}

}

state

2	3
5	6
8	9

from → 74

4 7

5 8

6 9

Unique Path II

Q. m x n grid

can move either down or right

Now, consider if some obstacles are added to the grids. How many unique paths could there be??

An obstacle is marked by 1

⊙		
	⊗	
		★

obstacle grid = $[[0,0,0],[0,1,0],[0,0,1]]$
Ans → 2

obstacleGrid = $[[0,1],[0,0]]$

⊙	⊗
	★

obstacle grid = $[[0,1],[0,0]]$
Ans → 1

int traverse (int i, int j, int m, int n, vector<vector<int>>& tab, vector<vector<int>>& obstacleGrid)

{

if (i < 0 && j < 0 && tab[i][j] != -1) return tab[i][j];

if (i < 0 && j < 0 && obstacleGrid[i][j] == 1) return 0;

if (i == 0 && j == 0) return tab[i][j] = 1;

if (i > 0 || j > 0) return 0;

return tab[i][j] = traverse(i-1, j, m, n, tab, Grid) + traverse(i, j-1, m, n, tab, Grid);

}

int UniquePath (vector<vector<int>>& Grid)

{

int m = Grid.size(); int n = Grid[0].size();

vector<vector<int>> tab(m+1, vector<int>(n+1, -1));

return traverse(1, 1, m, n, tab, obstacleGrid);

}

Minimum path sum;

1	3	1
1	5	1
4	2	1

grid = [[1,3,1], [1,5,1], [4,2,1]]

O/P:- 7

grid = [[1,2,3], [4,5,6]]

O/P:- 12

IF (i == m && j == n) { return grid[i-1][j-1]; }

IF (i > m || j > n) { return INT_MAX; }

Memorize by self
 int x = grid[i-1][j-1] + traverse(i+1, j, m, n, grid, tab)
 int y = grid[i-1][j+1] + traverse(i, j+1, m, n, grid, tab)
 return tab[i][j] = min(x, y)

Spiral Traversal

top = 0, down = N-1, left = 0, Right = N-1

0 →	1	2	3	4
5 ←	6	7	8	
9 →	10	11	12	
13 ←	14	15	16	

while (top <= down && left <= right)

{

if (dir == 0)

For i = left to Right

Print (a[top][i])

top++

else if (dir == 1)

For i = top to down

Print (a[i][right])

right--

ate

2	3
5	6
8	9

00 → 74

4 7

5 8

6 9

```

else IF ( dir == 2)
{
    For (i: Right to left)
        Print ( a[down][i] );
    down -= 1;
}

else IF ( dir == 3)
{
    For i = down to top
        Print ( a[i][left] );
    left += 1;
}

dir = (dir + 1) % 4
    
```

4 3 2 1

7 6 5 4

8 7 6 5

9 8 7 6

30

36

39

70

Date / /
Page

 Date / /
Page

Sparse Matrix and Polynomial Representation

Sparse Matrix $\rightarrow$ having more no. OF zero elements

There are two ways OF storing sparse Matrix to save memory as well as time.

1. Coordinate List / 3- Column representation
2. Compressed sparse row

	1	2	3	4	5	6	7	8	9
1	0	0	0	0	0	0	0	3	0
2	0	0	8	0	0	10	0	0	0
3	0	0	0	0	0	0	0	0	0
4	4	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0
6	0	0	2	0	0	0	0	0	0
7	0	0	0	6	0	0	0	0	0
8	0	9	0	0	5	0	0	0	0

8x9

5
0 0
5 0
0 0
0 4
5
4
5
4

array OF structure elements

1. 3 - column Representation

row	column	element
8	9	8
1	8	3
2	3	8
2	6	10
4	1	4
6	3	2
7	4	6
8	2	9
8	5	5

$\rightarrow$ First row is used to store no. OF rows & no. OF column & total no. OF non-zero element.

i, j, x

2. Compressed Sparse row

→ sparse matrix is represented using 3 arrays

		1	2	3	4	5	6	7	8	9
1	0	0	0	0	0	0	0	0	3	0
2	0	0	8	0	0	10	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0
4	4	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	0
6	0	0	2	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0	0
8	0	9	0	0	5	0	0	0	0	0

A [3, 8, 10, 4, 2, 6, 9, 5]

o index

1A [0, 1, 3, 3, 4, 4, 5, 6, 8]

indices → 0 1 2 3 4 5 6 7 8

JA [8, 3, 6, 1, 3, 4, 2, 5]

↓

$$8 + 9 + 8 = 25 \times 2 = 50 \text{ bytes}$$

$$7 \times 3 = 14 \text{ bytes}$$

↓
30% OF 14 bytes [so much memory saved]

Addition of Sparse Matrix

A	B	C
1 2 3 4 5 6	1 2 3 4 5 6	1 2 3 4 5 6
1 0 0 0 6 0 0	1 0 0 0 0 0 0	1 0 0 0 6 0 0
2 0 7 0 0 0 0	2 0 3 0 0 5 0	2 0 10 0 0 5 0
3 0 2 0 5 0 0	3 0 0 2 0 0 7	3 0 2 2 5 0 7
4 0 0 0 0 0 0	4 0 0 0 9 0 0	4 0 0 0 9 0 0
5 4 0 0 0 0 0	5 0 0 0 0 0 0	5 4 0 0 0 0 0
0 1 2 3 4 5	0 1 2 3 4 5 6	0 1 2 3 4 5 6 7 8 9
5 1 2 3 6 5	5 2 2 3 3 4 5	5 1 2 2 3 3 3 4 5
6 4 2 2 4 1	6 2 5 3 6 4 1	6 4 2 5 2 3 4 6 4 1
5 6 7 2 5 4	6 3 5 2 7 9 8	6 10 5 2 2 5 7 9 1

→ Add using 3 - column Representation.

Revise line by line
↓

Page No.

Date: / /

Array Representation of Sparse Matrix

1. How to Represent
2. How to create
3. How to add

struct Element

```
{
    int i;
    int j;
    int x;
};
```

struct sparse

```
{
    int m;
    int n;
    int num;
    struct Element *e;
};
```

	1	2	3	4	5
1	0	0	7	0	0
2	2	0	0	5	0
3	9	0	0	0	0
4	0	0	0	0	4

4x5 → m x n

	0	1	2	3	4	5
i	4	1	2	2	3	4
j	5	3	1	4	1	5
x	5	7	2	5	9	4

void create(struct sparse *s)

```
{
    int i;
```

printf("Enter Dimensions")

scanf("%d %d", &s->m, &s->n)

printf("Enter no of non-zero")

scanf("%d", &s->num)

s->e = new Element[s->num]

printf("Enter all elements")

for (i=0; i<s->m; i++)

scanf("%d %d %d", &s->e[i].i,

&s->e[i].j,

&s->e[i].x)

void main()

```
{
    struct sparse s;
```

create(&s);

display(s)

	0	1	2	3	4	5
i	4	1	2	2	3	4
j	5	3	1	4	1	5
x	5	7	2	5	9	4

	0	1	2	3	4	5
i						
j						
x						

void display(struct sparse s)

{ int i, j, k=0;

For (i=0; i<s->m; i++)

{

For (j=0; j<s->n; j++)

{

if (i==s->ele[k]-i & j==s->ele[k]-j)

printf("vd", s->ele[k++]-x);

else

printf("0");

printf("\n");

}

Enter Dimensions 5 5

Number of non-zero 5

Enter non-zero elements

0 0 1 1 0 0 0 0

1 0 1 1 0 0 0 0

2 0 1 1 0 0 0 0

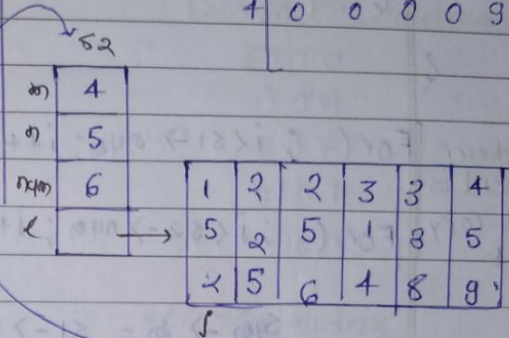
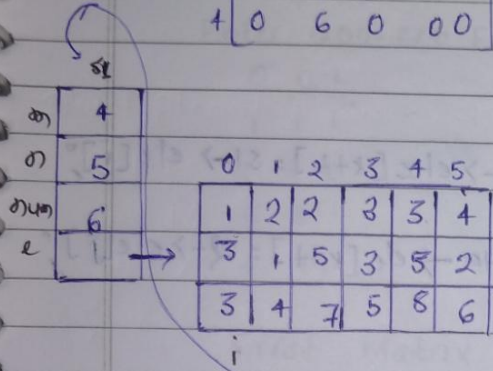
3 0 1 1 0 0 0 0

4 0 1 1 0 0 0 0

Program For Addition OF Sparse Matrices

	1	2	3	4	5
1	0	0	3	0	0
2	4	0	0	0	7
3	0	0	5	0	8
4	0	6	0	0	0

	1	2	3	4	5
1	0	0	0	0	2
2	0	5	0	0	6
3	4	0	8	0	0
4	0	0	0	0	9



struct sparse* add (sparse *s1, sparse *s2)
Also trace the program during revision
sparse *sum;

IF (s1->m != s2->m || s1->n != s2->n)
return 0;

sum = new sparse;

sum->m = s1->m; sum->n = s1->n;

sum->e = new Element [s1->m*n + s2->m*n];

While (i < s1->m*n & j < s2->m*n)

IF (s1->e[i].i < s2->e[j].i)

sum->e[k++] = s1->e[i++];

else IF (s1->e[i].i > s2->e[j].i)

sum->e[k++] = s2->e[j++];

Which ever is smaller copy it in sum
it is logic IF indices are equal then add it else

Revise thoroughly

printf ("sum Matrix\n");

display (*s3);

★ here send dereferenced value

Enter Dimensions

Number of non-zero

Enter non-zero Elements

0 0 1

1 1 1

2 2 1

3 3 1

4 4 1

Enter Dimensions

Number of non-zero

Enter non-zero elements

0 0 2

1 0 2

2 0 2

3 0 2

4 0 2

First Matrix

1 0 0 0 0

0 1 0 0 0

0 0 1 0 0

0 0 0 1 0

0 0 0 0 1

second Matrix

2 0 0 0 0

2 0 0 0 0

2 0 0 0 0

2 0 0 0 0

2 0 0 0 0

sum Matrix

3 0 0 0 0

2 1 0 0 0

2 0 1 0 0

2 0 0 1 0

2 0 0 0 1

Coding Sparse Matrix using C++

Class sparse

{

Private:

int m;

int n;

int num;

Element *ele;

Public:

Sparse(int m, int n, int num)

{

this->m = m;

this->n = n;

this->num = num;

ele = new Element[this->num];

}

~sparse()

{ delete [] ele;

}

void read()

{ cout << "Enter non-zero elements" << endl;

for (int i = 0; i < num; i++)

cin >> ele[i].i >> ele[i].j >> ele[i].x;

}

class Element

{

Public:

int i;

int j;

int x;

}

Page No. _____
 Date : / /

```

void display()
{
    int k = 0;
    For (int i = 0; i < 100; i++)
    {
        For (int j = 0; j < 10; j++)
        {
            IF (ele[k].i == i && ele[k].j == j)
                cout << ele[k].x << " ";
            else
                cout << "0 ";
        }
        cout << endl;
    }
}
  
```

Coding Sparse Matrix using insertion Operator

```
#include <iostream>
```

```
using namespace std;
```

```
class Element
```

```
{ public:
```

```
    int i;
```

```
    int j;
```

```
    int x;
```

```
};
```


Class Sparse

{ Private:

int m;

int n;

int num;

Element *ele;

Public:

Sparse(int m, int n, int num)

{

this->m = m;

this->n = n;

this->num = num;

ele = new Element[this->num];

}

~Sparse()

{

delete[] ele;

}

Sparse operator+(Sparse &s);

only one object
need to passed
as one is of
this class

friend istream & operator>> (istream &is, Sparse &s)

friend ostream & operator<< (ostream &os, Sparse &s)

{

Sparse Sparse::operator+(Sparse &s)

{ int i, j, k;

if (m == s.m || n == s.n)

return Sparse(0, 0, 0);

Page No. _____
Date: / /

```
sparse * sum = new sparse (m, n, num + s.num);
```

```
i = j = k = 0;
```

```
while (i < num && j < s.num)
```

```
{  
    if (ele[i].i < s.ele[j].i)
```

```
        sum->ele[k++] = ele[i++];
```

```
    else if (ele[i].i > s.ele[j].i)
```

```
        sum->ele[k++] = s.ele[j++];
```

```
    else  
    {
```

```
        if (ele[i].j < s.ele[j].j)
```

```
            sum->ele[k++] = ele[i++];
```

```
        else if (ele[i].j > s.ele[j].j)
```

```
            sum->ele[k++] = s.ele[j++];
```

```
        else
```

```
            sum->ele[k] = ele[i];
```

```
            sum->ele[k++].x = ele[i].x + s.ele[j].x;
```

```
    }  
    for (; i < num; i++) sum->ele[k++] = ele[i];
```

```
    for (; j < s.num; j++) sum->ele[k++] = s.ele[j];
```

```
    sum->num = k;
```

```
    return *sum;
```

→ dereferenced value

istream & operator >> (istream & is, sparse & s)

{

cout << "Enter non-zero elements";

for (int i = 0; i < s.m; i++)

cin >> s.ele[i].i >> s.ele[i].j >> s.ele[i].x;

return is;

}

ostream & operator << (ostream & os, sparse & s)

{

int k = 0;

for (int i = 0; i < s.m; i++)

{

for (int j = 0; j < s.n; j++)

{ if (s.ele[k].i == i && s.ele[k].j == j)

cout << s.ele[k].x << " ";

else

cout << "0 ";

k++;

cout << endl;

return os;

}

int main()

{ sparse s1(5,5,5);

sparse s2(5,5,5);

cin >> s1;

cin >> s2;

sparse sum = s1 + s2;

```

cout << "First Matrix" << endl << s1;
cout << "Second Matrix" << endl << s2;
cout << "sum Matrix" << endl << sum;
return 0;

```

Polynomial Representation

1. Polynomial Representation
2. Evaluation OF Polynomial
3. Addition OF Two Polynomial

$$P(x) = 3x^{\text{EXP}} + 2x^4 + 5x^2 + 2x + 7$$

↓
COEFF

1. Polynomial Representation :- contain OF two term COEFF & exponent

$$P(x) = 3x^5 + 2x^4 + 5x^2 + 2x + 7$$

$$n = 5$$

↓
no. OF term OR no. OF non-zero term

struct Term

struct poly

int COEFF;

int n;

int EXP;

struct Term *t

COEFF	3	2	5	2	7
EXP	5	4	2	1	0

struct poly P;

printf("No. of non-zero terms");

scanf("%d", &P.n);

n	5		0	1	2	3	4
t		→ Coeff					
		exp					

P.t = new Term[P.n];

printf("Enter Polynomial terms");

for(i=0; i<P.n; i++)

{

printf("Terms no. %d", i+1);

scanf("%d %d", &P.t[i].coeff, &P.t[i].exp);

}

2. Evaluation of Polynomial

$$P(x) = 3x^5 + 2x^4 + 5x^3 + 2x + 7$$

struct Term

{ int coeff;

int exp;

};

struct poly

{ int n;

struct Term *t;

};

struct Poly P;

sum = 0;

for(i=0; i<P.n; i++)

{

sum += P.t[i].coeff * pow(x, P.t[i].exp);

}

return sum;

n	5		0	1	2	3	4
t		→ Coeff	3	2	5	2	7
		exp	5	4	3	1	0

3. Addition OF Two polynomial

$$P_1(x) = 5x^4 + 2x^2 + 5$$

$$P_2(x) = 6x^4 + 5x^3 + 4x^2 + 2x + 3$$

PL

0	3		0	1	2
1		Cost	5	2	5
		Exp	+	2	0

i

P_2
 1)

5
—

 $\xrightarrow{\text{swap}}$

6	5	9	2	5
4	5	2	1	0

 Exp

	P_3		0	1	2	3	4
α_j		Cost	11	15	11	2	8
t	$\rightarrow \text{exp}$		4	3	2	1	0

$$i=0, j=0, K=0$$

While ($i < P1.n$ && $j < P2.n$)

28

$$IF(P1.t[i].exp \neq P2.t[j].exp)$$
$$P3.t[K+H] = P10.t[i+H];$$

else if ($P_2.t[i].exp > P_1.t[i].exp$)

$$P_3 \cdot t[k+t] = P_2 \cdot t[j+t];$$

else

$$\S \quad p_{\beta_0} \in [K] \cdot \text{EXP} = p_1 \cdot t[i] \cdot \text{EXP}$$

P3. $E[K^{++}] \cdot \text{COEFF} = P1 \cdot E[I^{++}] \cdot \text{COEFF} +$

$$P_{2n} t[j+1].\text{coeff}$$

4

→ #include <stdio.h>
#include <stdlib.h>

```

struct Term
{
    int coeff;
    int exp;
};

struct poly
{
    int n;
    struct Term * terms;
};

```

```

void create(struct poly *P)
{
    int i;
    printf("Number of terms?");
    scanf("%d", &P->n);
    P->terms = (struct Term*) malloc(P->n * sizeof(struct Term));
    printf("Enter terms\n");
    for(i = 0; i < P->n; i++)
        scanf("%d %d", &P->terms[i].coeff, &P->terms[i].exp);
}

```

```

void display(struct poly P)
{
    int i;
    for(i = 0; i < P.n; i++)
        printf("%d x %d", P.terms[i].coeff, P.terms[i].exp);
    printf("\n");
}

```

```

struct poly *add(struct poly *P1, struct poly *P2)
{
    int i, j, k;
    struct poly *sum;
    sum = (struct poly*) malloc(sizeof(struct poly));
}

```

$\rightarrow$ sum $\rightarrow$ terms = (struct Term *) malloc((P1 $\rightarrow$ n + P2 $\rightarrow$ n) * sizeof(struct Term))
 $i = j = k = 0;$

while (i < P1 $\rightarrow$ n && j < P2 $\rightarrow$ n)

{

if (P1 $\rightarrow$ terms[i].exp > P2 $\rightarrow$ terms[j].exp)

sum $\rightarrow$ terms[k++] = P1 $\rightarrow$ terms[i++];

else if (P1 $\rightarrow$ terms[i].exp < P2 $\rightarrow$ terms[j].exp)

sum $\rightarrow$ terms[k++] = P2 $\rightarrow$ terms[j++];

else

{ sum $\rightarrow$ terms[k].exp = P1 $\rightarrow$ terms[i].exp;

sum $\rightarrow$ terms[k].coeff = P1 $\rightarrow$ terms[i].coeff
+ P2 $\rightarrow$ terms[j].coeff;

}

for (; i < P1 $\rightarrow$ n; i++) sum $\rightarrow$ terms[k++] = P1 $\rightarrow$ terms[i];

for (; j < P2 $\rightarrow$ n; j++) sum $\rightarrow$ terms[k++] = P2 $\rightarrow$ terms[j];

sum $\rightarrow$ n = k;

return sum;

{

int main()

{ struct poly p1, p2, *p3;

create(&p1);

create(&p2);

p3 = add(&p1, &p2);

printf("\n");

display(p1);

printf("\n");

display(p2);

printf("\n");

return; display(*p3);

{

deferred

Number of terms?

Enter terms

1 5

1 3

1 1

Number of terms?

enter terms

1 4

1 2

1 0

1x5 + 1x3 + 1x1 +

1x4 + 1x2 + 1x0 +

1x5 + 1x4 + 1x3 + 1x2 + 1x1 + 1x0 +

→ swap → swap = (swap temp) swap((f1, f2) + (f2, f1))

while (i < f1) { f1 = f1 + 1; f2 = f2 - 1; }

if (f1 > f2) { swap(f1, f2); f1 = f1 + 1; f2 = f2 - 1; }

swap → swap[k+1] = f1 → swap[k+1]

if (f1 > f2) { swap(f1, f2); f1 = f1 + 1; f2 = f2 - 1; }

swap → swap[k+1] = f2 → swap[k+1]

swap → swap[k] = f1 → swap[k]

swap → swap[k+1] = f2 → swap[k+1]

for (i < f1; i < f2; i++) { swap(f1, f2); f1 = f1 + 1; f2 = f2 - 1; }

for (i < f2; i < f1; i++) { swap(f1, f2); f1 = f1 + 1; f2 = f2 - 1; }

swap → swap = k

return swap;

int main()

int arr[10];

create(arr);

create(arr);

ps = add(arr, arr);

printf("%d", ps);

display(arr);

printf("%d", ps);

display(arr);

printf("%d", ps);

display(arr);

printf("%d", ps);

display(arr);

printf("%d", ps);

display(arr);

printf("%d", ps);

display(arr);

printf("%d", ps);

display(arr);